



SESSION 2

NODE.JS ESSENTIALS

Mục tiêu học tập

Trong buổi học này, học sinh sẽ học cách:

- Mô tả kiến trúc của Node.js

- Liệt kê các lợi ích và hạn chế của ứng dụng Node.js

- Giải thích khái niệm về ứng dụng dựa trên bảng điều khiển trong Node.js

- Giải thích các cấu trúc lập trình chính trong Node.js

- So sánh vai trò của Node Package Manager (NPM) và Node Version

 - Trình quản lý (NVM) trong Node.js

- Mô tả các đối tượng toàn cục và cách sử dụng chúng trong các ứng dụng Node.js

Node.js là một môi trường phía máy chủ sử dụng JavaScript để thực hiện các thao tác yêu cầu-phản hồi. Nó có những ưu điểm và hạn chế riêng trong lĩnh vực điện toán. Môi trường dòng lệnh tương tác và không tương tác rất phổ biến trong Node.js. Các yếu tố lập trình thiết yếu như biến, biểu thức đa dòng, kiểu dữ liệu, hàm và mảng là một phần của quá trình phát triển ứng dụng Node.js. Trong Node.js, Trình quản lý gói Node (NPM) và Trình quản lý phiên bản Node (NVM) cung cấp thông tin về các gói và phiên bản tương ứng.

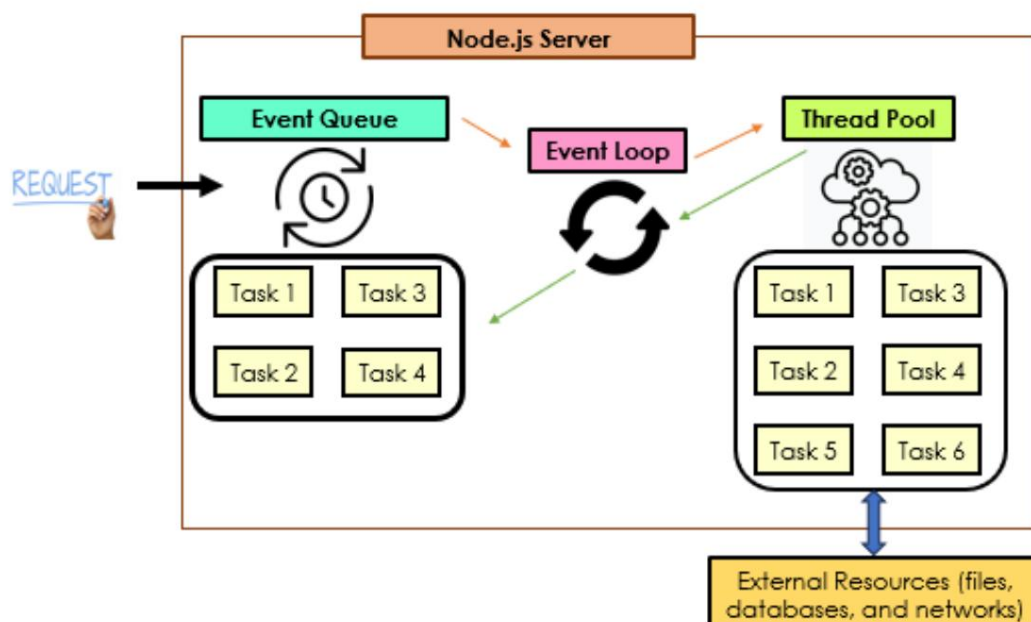
Buổi học này sẽ trình bày chi tiết về kiến trúc của Node.js, bao gồm các thành phần và quy trình làm việc. Buổi học cũng sẽ khám phá những lợi ích và hạn chế của các ứng dụng Node.js. Buổi học cũng sẽ mô tả khái niệm về một ứng dụng dựa trên bảng điều khiển.

ứng dụng và môi trường Đọc In Đánh giá Vòng lặp (REPL). Nó cũng sẽ Giải thích các cấu trúc lập trình quan trọng, chẳng hạn như kiểu dữ liệu, hàm và mảng, được sử dụng trong các ứng dụng Node.js. Cuối cùng, buổi học sẽ kết thúc bằng việc so sánh vai trò của NPM và NVM trong Node.js và cách sử dụng các đối tượng toàn cục trong các ứng dụng Node.js.

2.1 Giới thiệu về kiến trúc Node.js

Node.js là một môi trường chạy hiệu quả dựa trên JavaScript. Đây là một nền tảng đơn luồng, trong đó một luồng chính duy nhất xử lý tất cả các yêu cầu từ máy khách. Node.js được phát triển trên nền tảng JavaScript V8 của Google Chrome.

Hình 2.1 mô tả biểu diễn trực quan của kiến trúc Node.js.



Hình 2.1: Kiến trúc Node.js

2.1.1 Các thành phần của kiến trúc Node.js

Các thành phần có trong kiến trúc Node.js như sau:



2.1.2 Quy trình làm việc của Node.js

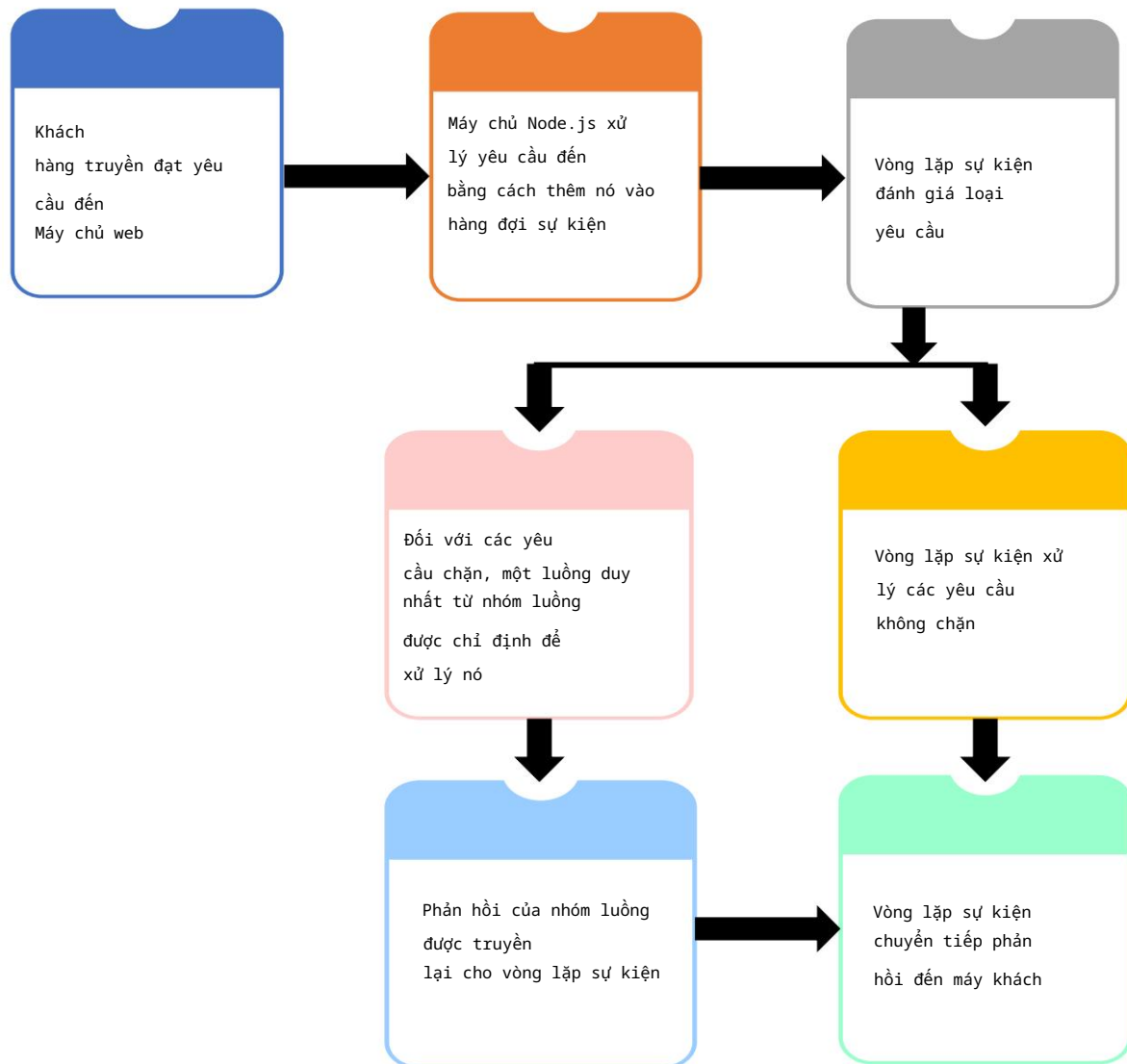
Node.js áp dụng chế độ thực thi đồng thời. Điều này có nghĩa là nó không đợi tác vụ hiện tại kết thúc rồi mới bắt đầu thực thi tác vụ tiếp theo. Sẽ rất lãng phí tài nguyên CPU nếu Node.js chờ mỗi tác vụ kết thúc trước khi tiếp tục tác vụ tiếp theo.

Khi máy khách gửi yêu cầu đến máy chủ, một luồng sẽ được gán cho yêu cầu đó. Luồng này chịu trách nhiệm xử lý yêu cầu từ đầu đến cuối.

Luồng chứa các hướng dẫn cụ thể để xử lý yêu cầu và tạo phản hồi. Phản hồi do luồng tạo ra sẽ được gửi đến vòng lặp sự kiện.

Vòng lặp sự kiện có nhiệm vụ truyền phản hồi trở lại cho máy khách.

Sau đây là hình ảnh minh họa quy trình làm việc của Node.js.



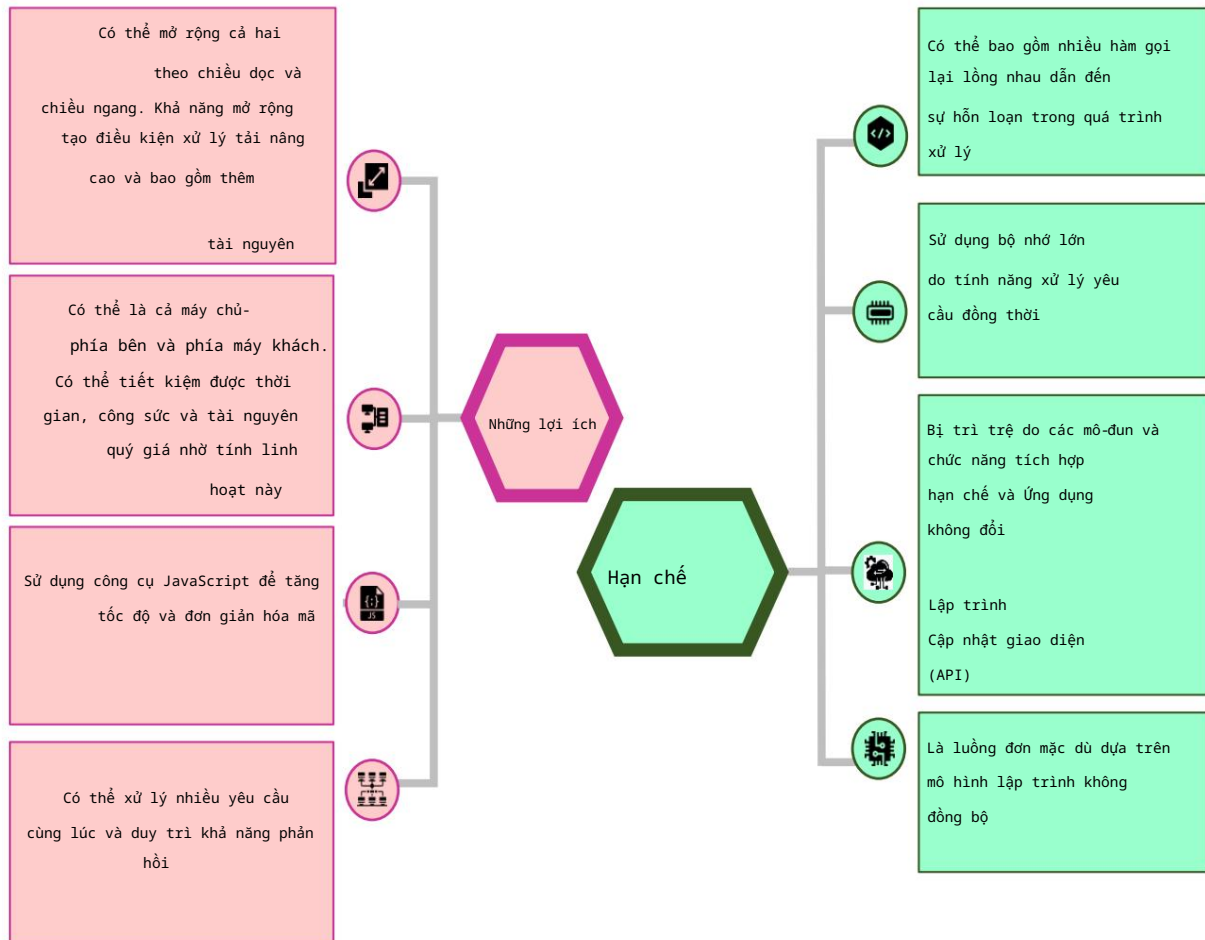
Một tập hợp các hướng dẫn cụ thể tới máy chủ được gọi là luồng trong Node.js.

2.2 Lợi ích và hạn chế của Node.js

Các nhà phát triển phần mềm thành thạo JavaScript sẽ thấy việc hiểu Node.js trong ứng dụng của họ khá dễ dàng. Họ có thể phát triển chương trình bằng JavaScript cho cả ứng dụng front-end và back-end. Tuy nhiên, Node.js cũng có những hạn chế riêng.

Lợi ích và hạn chế của ứng dụng Node.js được trình bày ở đây.

Các ứng dụng Node.js:



2.3 Ứng dụng dựa trên bảng điều khiển trong Node.js

Có hai cách để phát triển ứng dụng Node.js: dựa trên console và dựa trên web. Ứng dụng Node.js dựa trên console chạy tại dấu nhắc lệnh. Các ứng dụng này không có Giao diện người dùng đồ họa (GUI).

Mô-đun console trong Node.js có hàm `console.log` giúp hiển thị các thông báo tùy chỉnh trên thiết bị đầu cuối VS Code hoặc dấu nhắc lệnh.

Hãy xem xét một mã ví dụ trong đó một thông báo tùy chỉnh sử dụng `console.log` chức năng sẽ được hiển thị trên thiết bị đầu cuối.

Thực hiện các bước sau:

1. Mở tệp văn bản trong VS Code.
2. Nhập mã được cung cấp vào tệp.

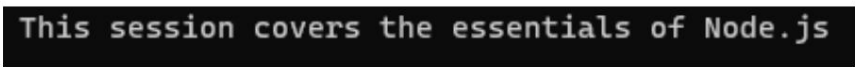
```
console.log('Phiên này bao gồm những nội dung cơ bản của Node.js');
```

Trong mã ví dụ, một thông báo tùy chỉnh được cung cấp làm đối số cho hàm `console.log`.

3. Lưu tệp dưới dạng `sample_display.js` trong một thư mục trên hệ thống cục bộ.
4. Mở cửa sổ dòng lệnh và điều hướng đến thư mục cần thiết. Hoặc, mở VS Code Terminal. Đảm bảo biến đường dẫn môi trường bao gồm đường dẫn cài đặt Node.js.
5. Để chạy chương trình đã lưu, tại dấu nhắc lệnh hoặc VS Code Terminal, nhập lệnh sau và nhấn Enter.

```
nút sample_display.js
```

Hình 2.2 hiển thị đầu ra của mã.



```
This session covers the essentials of Node.js
```

Hình 2.2: Đầu ra của bảng điều khiển

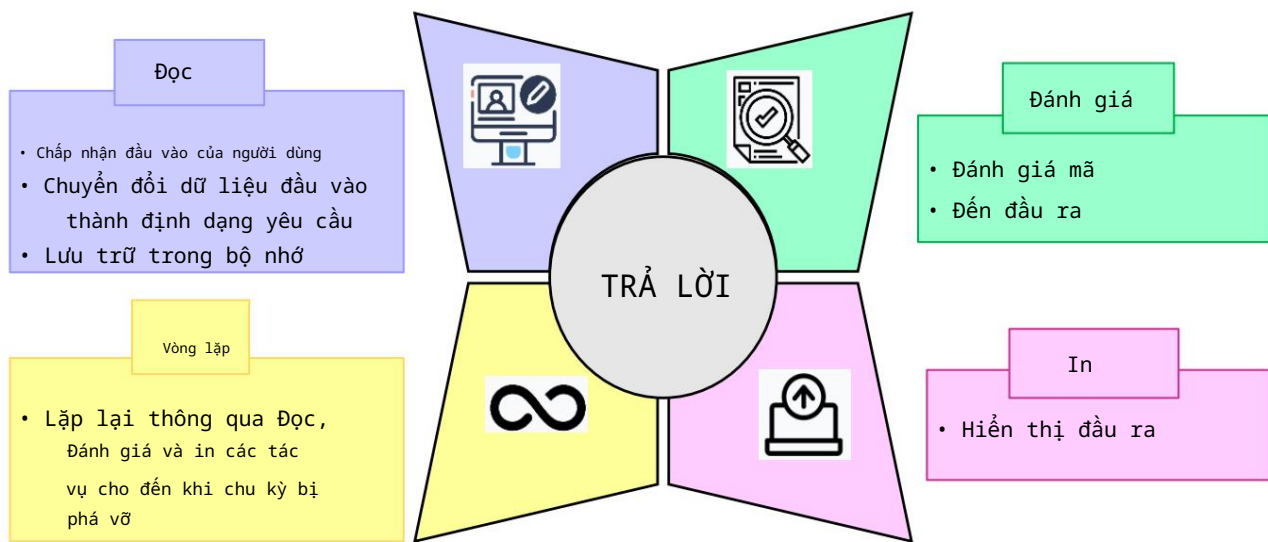
Quan sát đầu ra của chương trình. Thông báo tùy chỉnh sẽ được hiển thị trên dấu nhắc lệnh hoặc Terminal.

2.3.1 Tổng quan về REPL

Trong Node.js, REPL là viết tắt của Read Evaluate Print Loop (Đọc Đánh giá Vòng lặp In). REPL tương tự như môi trường dòng lệnh Windows hoặc shell Unix/Linux, được cài đặt cùng với Node.js trên hệ thống.

Cách thông thường để thực thi một chương trình Node.js là chạy `.js` đã lưu tệp tại dấu nhắc lệnh. Tuy nhiên, khi sử dụng REPL, các đoạn mã nhỏ có thể được thực thi trực tiếp tại dấu nhắc REPL mà không cần tạo tệp `.js`.

Mỗi phần trong bốn phần của REPL đều đóng một vai trò cụ thể. Nhiệm vụ của bốn phần này như sau:



Để kích hoạt dấu nhắc REPL trong Windows, hãy thực hiện các bước sau:

1. Mở cửa sổ dấu nhắc lệnh.
2. Tại dấu nhắc lệnh, nhập lệnh cho sẵn và nhấn Enter.

nút

Hình 2.3 hiển thị đầu ra lệnh tại dấu nhắc lệnh.

```
Welcome to Node.js v21.1.0.  
Type ".help" for more information.  
>
```

Hình 2.3: Đầu ra lệnh để khởi chạy REPL

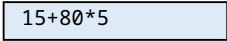
3. Dấu nhắc lệnh REPL xuất hiện.
4. Bắt đầu thực thi các lệnh Node.js.

Làm việc với các biểu thức đơn giản

Các biểu thức toán học đơn giản có thể được thực hiện trực tiếp tại dấu nhắc lệnh REPL.

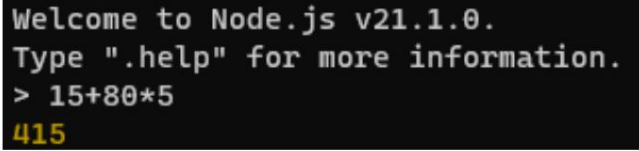
Hãy xem xét một ví dụ để hiểu rõ hơn. Thực hiện các bước sau:

1. Kích hoạt dấu nhắc REPL trong Windows.
2. Nhập biểu thức toán học tại dấu nhắc REPL và nhấn Enter.



15+80*5

Hình 2.4 hiển thị kết quả của biểu thức toán học.



```
Welcome to Node.js v21.1.0.
Type ".help" for more information.
> 15+80*5
415
```

Hình 2.4: Đầu ra của biểu thức toán học

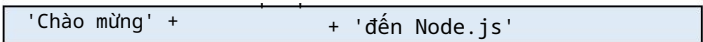
Quan sát đầu ra. Kết quả của biểu thức toán học được hiển thị trên terminal. Chưa có tệp .js riêng biệt nào được tạo để lưu trữ hoặc xử lý biểu thức toán học.

Tương tự như các biểu thức toán học đơn giản, biểu thức chuỗi cũng có thể được thực thi tại dấu nhắc lệnh REPL.

Hãy xem xét một ví dụ trong đó một biểu thức chuỗi đơn giản sử dụng toán tử nối được thực thi tại dấu nhắc lệnh REPL.

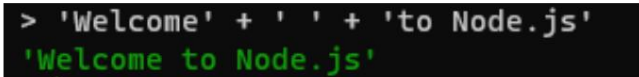
Thực hiện các bước sau:

1. Kích hoạt dấu nhắc REPL trong Windows.
2. Nhập biểu thức chuỗi đã cho tại dấu nhắc REPL và nhấn Enter.



'Chào mừng' + ' ' + 'đến Node.js'

Hình 2.5 hiển thị kết quả đầu ra của biểu thức chuỗi.



```
> 'Welcome' + ' ' + 'to Node.js'
'Welcome to Node.js'
```

Hình 2.5: Đầu ra của biểu thức chuỗi

Hãy quan sát kết quả đầu ra. Toán tử nối sẽ nối hai chuỗi và tạo ra một chuỗi duy nhất. Kết quả của biểu thức chuỗi được hiển thị trên màn hình.

Làm việc với các biến

Biến là một thực thể lập trình cung cấp tên cho một vị trí cụ thể trong bộ nhớ hệ thống. Một giá trị có thể được lưu trữ trong một biến và giá trị đó có thể được

được sử dụng trong các chương trình thông qua biến. Trong mã Node.js, sử dụng từ khóa `var` để khai báo và gán giá trị cho biến.

Hãy xem xét một ví dụ trong đó các biến được sử dụng để minh họa phép cộng.

Thực hiện các bước sau:

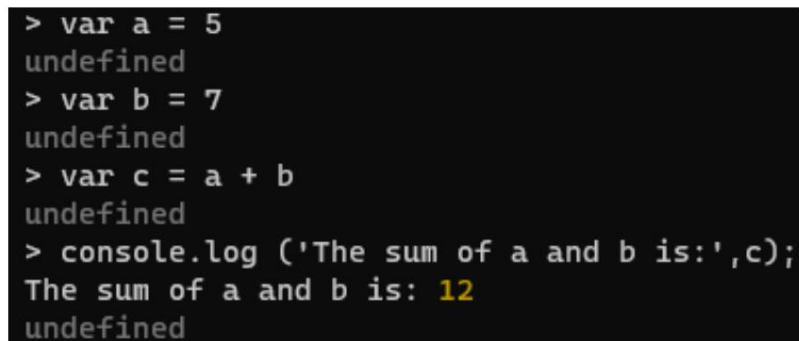
1. Kích hoạt dấu nhắc REPL trong Windows.
2. Nhập từng câu lệnh được đưa ra trong Đoạn mã 1 tại dấu nhắc REPL. Đảm bảo nhấn Enter ở cuối mỗi câu lệnh để thực thi.

Đoạn mã 1:

```
biến a = 5  
biến b = 7  
var c = a + b  
console.log('Tổng của a và b là:',c);
```

Trong Đoạn mã 1, các giá trị được lưu trữ trong các biến `a` và `b` tương ứng. Biến `c` lưu trữ tổng của các biến `a` và `b`. Kết quả của phép cộng được hiển thị dưới dạng đầu ra.

Hình 2.6 hiển thị kết quả của phép cộng.



```
> var a = 5  
undefined  
> var b = 7  
undefined  
> var c = a + b  
undefined  
> console.log ('The sum of a and b is:',c);  
The sum of a and b is: 12  
undefined
```

Hình 2.6: Đầu ra của phép cộng sử dụng biến

Quan sát kết quả đầu ra: Tổng của 5 và 7 là 12.

Lưu ý rằng theo mặc định, giá trị trả về của các lệnh được hiển thị là `undefined` sau khi thực thi mỗi lệnh. Điều này có thể được loại bỏ bằng cách đặt tùy chọn `ignoreUndefined` thành `true` khi khởi động một phiên bản REPL như được chỉ ra trong lệnh sau:

```
node -e "require('repl').start({ignoreUndefined: true})"
```

Giá trị mặc định là sai.

Làm việc với biểu thức đa dòng

Môi trường REPL có khả năng nhận dạng các biểu thức hoàn chỉnh ngay tại dấu nhắc. Nếu nhấn phím Enter ở cuối một biểu thức chưa hoàn chỉnh, REPL sẽ tự động thêm dấu ba chấm (.) vào dòng tiếp theo. Dấu ba chấm này báo hiệu tiếp tục mã.

Hãy xem xét một ví dụ minh họa việc sử dụng biểu thức đa dòng REPL với sự trợ giúp của vòng lặp do-while.

Thực hiện các bước sau:

1. Kích hoạt dấu nhắc REPL trong Windows.
2. Nhập từng câu lệnh đã cho trong Đoạn mã 2 tại dấu nhắc REPL. Đảm bảo nhấn Enter ở cuối mỗi câu lệnh để đảm bảo tính liên tục.

Đoạn mã 2:

```
biến a = 5
biến b = 7
biến i = 0
LÀM{
    var c = a + b
    tôi++
    một++
    b++
    console.log ('Tổng của a và b trong lần lặp',i,'is',
        c)
}trong khi (i<3)
```

Trong Đoạn mã 2, các biến a, b và i được khai báo và gán giá trị.

Bên trong vòng lặp do-while:

- Các biến a và b được thêm vào.
- Kết quả được lưu trữ trong biến c.
- Các giá trị của a, b và i được tăng thêm 1.
- Kết quả của phép cộng thu được trong mỗi lần lặp được hiển thị trên phần cuối.

Vòng lặp do-while tiếp tục thực thi cho đến khi giá trị của i nhỏ hơn 3.

Hình 2.7 hiển thị kết quả của các biểu thức đa dòng.

```

> var a = 5
undefined
> var b = 7
undefined
> var i = 0
undefined
> do{
... var c = a + b
... i++
... a++
... b++
... console.log ('Sum of a and b in iteration', i, 'is', c)
... }while (i<3)
Sum of a and b in iteration 1 is 12
Sum of a and b in iteration 2 is 14
Sum of a and b in iteration 3 is 16
undefined

```

Hình 2.7: Đầu ra của biểu thức đa dòng

Quan sát kết quả đầu ra. Dấu ba chấm sẽ tự động được tạo ra để tiếp tục mã. Kết quả của các lần lặp lại được hiển thị là 12, 14 và 16.

Bảng 2.1 liệt kê một số lệnh thường dùng trong Node.js REPL.

Lệnh	Sự miêu tả
Ctrl + C/Ctrl + c	Dừng thực thi lệnh hiện tại
Ctrl + C hai lần/Ctrl + c hai lần/Ctrl + d	Thoát khỏi môi trường REPL
phím lên/xuống	Giúp kiểm tra lịch sử lệnh và cập nhật các lệnh trước đó
phím tab	Hiển thị danh sách các lệnh hiện tại
.giúp đỡ	Hiển thị danh sách tất cả các lệnh có sẵn
.phá vỡ/.xóa	Thoát khỏi biểu thức đa dòng
.save <tên tệp>	Lưu phiên REPL hiện tại vào một tệp được cung cấp trong lệnh
.load <tên tệp>	Tải nội dung tệp đã chỉ định vào tệp hiện tại Phiên REPL

Bảng 2.1: Lệnh REPL

2.4 Cấu trúc lập trình Node.js

Có nhiều loại cấu trúc lập trình khác nhau trong Node.js. Những cấu trúc này cho phép các nhà phát triển tạo ra nhiều dạng ứng dụng khác nhau.

Một số cấu trúc lập trình trong Node.js như sau:

Kiểu dữ liệu

Chức năng

Mảng

2.4.1 Tổng quan về các kiểu dữ liệu

Kiểu dữ liệu trong ngôn ngữ lập trình là những thực thể quan trọng. Kiểu giá trị được lưu trữ trong một biến biểu thị tầm quan trọng của kiểu dữ liệu đó. Biến có thể lưu trữ nhiều kiểu giá trị khác nhau như văn bản, số, thập phân, v.v.

Đối với mỗi biến, có thể gán một kiểu dữ liệu.

Sau đây là một số kiểu dữ liệu khác nhau có thể được sử dụng trong các ứng dụng Node.js.



Làm việc với Kiểu dữ liệu số

Kiểu dữ liệu số có thể nhận cả giá trị dương và âm. Hãy xem xét đoạn mã ví dụ trong Đoạn mã 3, trong đó các biến được khai báo bằng kiểu dữ liệu số và các phép toán số học được minh họa.

Đoạn mã 3:

```
biến a = 40;  
biến b = 10,5;
```

```

biến c = -4;
console.log('Giá trị của a là:',a);
console.log('Giá trị của b là:',b);
console.log('Giá trị của c là:',c);
console.log('Tổng của a và c là:' (a+c));
console.log('Trừ b khỏi c là:' (c - b));
console.log('Tích của c và b là : ' console.log('Thương của a/b là : ' (a / b));
console.log('Phần còn lại của a/b là : ' (a % b));

```

Trong Đoạn mã 3, mặc dù var được sử dụng, các biến a, b và c được suy ra là thuộc kiểu dữ liệu số dựa trên nội dung chúng chứa. Nhiều phép toán khác nhau như cộng, trừ, nhân và chia được thực hiện trong chương trình.

Hình 2.8 hiển thị kết quả của Đoạn mã 3.

```

Value of a is: 40
Value of b is: 10.5
Value of c is: -4
Sum of a and c is : 36
Subtracting b from c is: -14.5
Product of c and b is : -42
Quotient of a/b is : 3.8095238095238093
Remainder of a/b is : 8.5

```

Hình 2.8: Đầu ra của đoạn mã 3

Quan sát đầu ra của chương trình. Kết quả của các phép toán được hiển thị trên bảng điều khiển.

Làm việc với kiểu dữ liệu String

Để gán giá trị cho biến kiểu dữ liệu chuỗi, hãy sử dụng dấu ngoặc đơn hoặc dấu ngoặc kép. Biến chỉ trở thành biến chuỗi sau khi giá trị theo định dạng yêu cầu được gán cho nó.

Hãy xem xét một đoạn mã ví dụ trong Đoạn mã 4, trong đó minh họa cách nối chuỗi.

Đoạn mã 4:

```

var str1 = 'Chào mừng ';
var str2 = 'đến Phiên Node.js 2';
console.log('Chuỗi đầu tiên là:',str1);
console.log('Chuỗi thứ hai là:',str2);
console.log('Chuỗi nối là:',str1+str2);

```

Trong Đoạn mã 4, hai biến chuỗi được khai báo. Hai chuỗi này được nối với nhau bằng toán tử nối +.

Hình 2.9 hiển thị kết quả của Đoạn mã 4.

```
First string is: Welcome
Second string is: to Node.js Session 2
Concatenated string is: Welcome to Node.js Session 2
```

Hình 2.9: Đầu ra của đoạn mã 4

Quan sát đầu ra của chương trình. Chuỗi đầu tiên, chuỗi thứ hai và chuỗi được nối sẽ được hiển thị.

Làm việc với Kiểu dữ liệu Boolean

Kiểu dữ liệu Boolean giúp các nhà phát triển xây dựng logic so sánh và có điều kiện trong các ứng dụng Node.js.

Hãy xem xét một ví dụ mã trong Đoạn mã 5, trong đó các biến kiểu dữ liệu Boolean được minh họa.

Đoạn mã 5:

```
var i = sai;
biến a = 6;
biến b = 9;

console.log('Giá trị Boolean của biến i là:', i);
console.log('Giá trị của biến a là:', a);
console.log('Giá trị của biến b là:', b);
console.log('Giá trị của biến a và b giống nhau:', a == b);

console.log('Giá trị của biến a không giống nhau
như giá trị của biến b:', a != b);
console.log('Giá trị Boolean của !i là:', !i);
```

Trong Đoạn mã 5, biến i được khai báo bằng kiểu dữ liệu Boolean. Các toán tử như bằng, không bằng và không được sử dụng để so sánh các giá trị.

Hình 2.10 hiển thị kết quả của Đoạn mã 5.

```
The Boolean value of variable i is: false
The value of variable a is: 6
The value of variable b is: 9
The values of variables a and b are the same: false
The value of variable a is not the same as value of variable b: true
The Boolean value of !i is: true
```

Hình 2.10: Đầu ra của đoạn mã 5

Quan sát đầu ra của chương trình. Các giá trị Boolean được hiển thị là đúng hoặc SAI.

Làm việc với kiểu dữ liệu chưa xác định

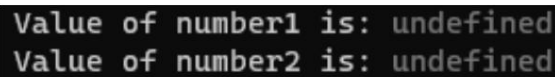
Hãy xem xét một ví dụ mã trong Đoạn mã 6, trong đó các biến kiểu dữ liệu chưa xác định được minh họa.

Đoạn mã 6:

```
biến số 1;  
console.log('Giá trị của number1 là:',number1);  
biến số 2;  
console.log('Giá trị của number2 là:',number2);
```

Trong Đoạn mã 6, hai biến được khai báo mà không có giá trị nào.

Hình 2.11 hiển thị kết quả của Đoạn mã 6.



```
Value of number1 is: undefined  
Value of number2 is: undefined
```

Hình 2.11: Đầu ra của đoạn mã 6

Quan sát kết quả đầu ra của chương trình. Vì không có giá trị nào được gán cho các biến nên kết quả đầu ra là undefined.

Làm việc với kiểu dữ liệu Null

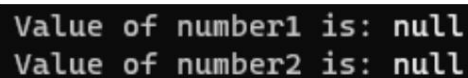
Hãy xem xét một ví dụ mã trong Đoạn mã 7, trong đó các biến kiểu dữ liệu null được minh họa.

Đoạn mã 7:

```
biến số1 = null;  
console.log("Giá trị của number1 là:",number1);  
var number2 = null;  
console.log("Giá trị của number2 là:",number2);
```

Trong Đoạn mã 7, hai biến được khai báo và gán giá trị null.

Hình 2.12 hiển thị kết quả của Đoạn mã 7.



```
Value of number1 is: null  
Value of number2 is: null
```

Hình 2.12: Đầu ra của đoạn mã 7

Quan sát đầu ra của chương trình. Các giá trị được hiển thị là null.

Làm việc với Kiểu dữ liệu đối tượng

Tên và giá trị đóng vai trò quan trọng trong kiểu dữ liệu đối tượng. Mỗi biến được khai báo dưới dạng đối tượng sẽ có cặp tên:giá trị. Tên phải là duy nhất.

Mỗi tên sẽ có một giá trị tương ứng. Tuy nhiên, các giá trị có thể được lặp lại. Dấu phẩy phân tách các cặp tên:giá trị và chúng được trình bày trong dấu ngoặc nhọn. Một thuật ngữ khác cho cặp tên:giá trị là thuộc tính đối tượng.



Cặp tên:giá trị cho kiểu dữ liệu đối tượng còn được gọi là cặp khóa:giá trị trong Node.js.

Hãy xem xét một ví dụ mã trong Đoạn mã 8, trong đó kiểu dữ liệu đối tượng được minh họa.

Đoạn mã 8:

```
var sinh viên = {  
  Tên: 'David Franklin',  
  Tuổi: 12,  
  Lớp: 8  
};  
console.log('Tên:',tên học sinh);  
console.log('Tuổi:',tuổi của học sinh);  
console.log('Điểm:',học sinh.Điểm);
```

Trong Đoạn mã 8, biến student được khai báo là một kiểu dữ liệu đối tượng với cặp khóa:giá trị. Khóa là Name, Age và Grade. Giá trị là David. Franklin, 12 tuổi và 8 tuổi.

Hình 2.13 hiển thị kết quả của Đoạn mã 8.

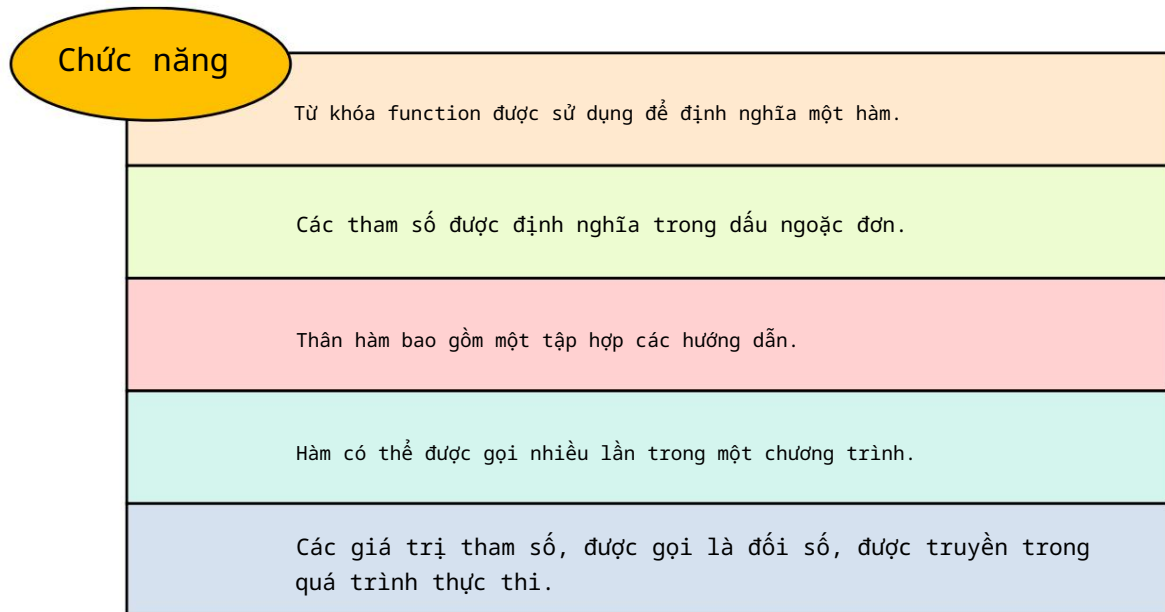
```
Name: David Franklin  
Age: 12  
Grade: 8
```

Hình 2.13: Đầu ra của đoạn mã 8

Quan sát đầu ra của chương trình. Các khóa và giá trị của đối tượng được hiển thị.

2.4.2 Tổng quan về các chức năng

Hàm là một cấu trúc lập trình quan trọng trong Node.js. Có hai loại hàm: hàm tích hợp sẵn và hàm do người dùng định nghĩa.



Hãy xem xét một đoạn mã ví dụ trong Đoạn mã 9, trong đó một hàm do người dùng định nghĩa được tạo và thực thi.

Đoạn mã 9:

```
hàm học sinh (tên, tuổi, lớp, điểm 1, điểm 2) {  
    console.log('Tên:', tên);  
    console.log('Tuổi:', tuổi);  
    console.log('Điểm:', điểm);  
    var total_marks=mark1+mark2;  
    console.log('Tổng điểm:', total_marks)  
}  
sinh viên('David Franklin', 13, 8, 78, 85);
```

Trong Đoạn mã 9, một hàm do người dùng định nghĩa có tên là student đã được định nghĩa.

Hàm này tính tổng điểm của một học sinh và có năm tham số.

Giá trị cho các tham số này được truyền dưới dạng đối số khi hàm được gọi.

Hình 2.14 hiển thị kết quả của Đoạn mã 9.

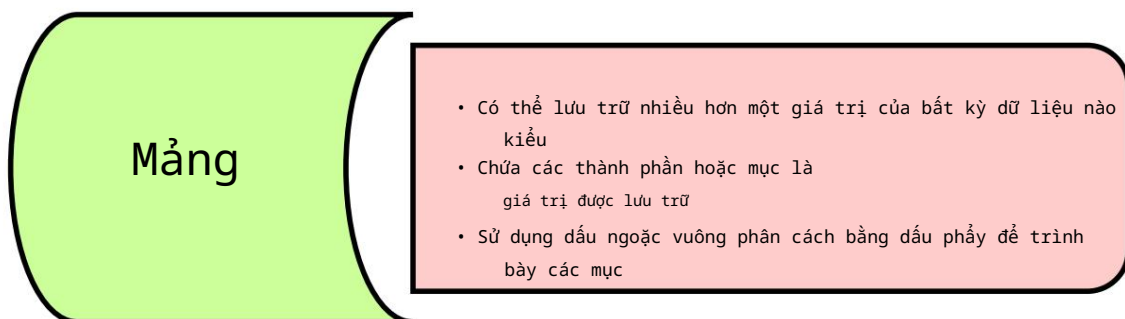
```
Name: David Franklin
Age: 13
Grade: 8
Total Marks: 163
```

Hình 2.14: Đầu ra của đoạn mã 9

Quan sát kết quả đầu ra của chương trình. Tên, tuổi, điểm và điểm số được truyền vào làm đối số khi hàm được gọi. Tổng điểm được tính toán và giá trị của đối tượng được hiển thị.

2.4.3 Tổng quan về Mảng

Mảng là một cấu trúc lập trình thường được sử dụng trong Node.js.



Hãy xem xét một ví dụ mã trong Đoạn mã 10, trong đó một mảng được khai báo và minh họa.

Đoạn mã 10:

```
biến arr1 = [76,80,85,90,92];
var tổng=0, trung bình=0;
đối với (var i = 0; i < arr1.length; i++) {
    tổng=tổng+arr1[i]
    avg=tổng/5
}
console.log('Điểm trung bình của năm môn:',avg);
```

Trong Đoạn mã 10, một mảng được khai báo để lưu trữ điểm của năm môn học mà một học sinh đạt được. Tổng điểm và điểm trung bình của một học sinh được tính toán.

Hình 2.15 hiển thị kết quả của Đoạn mã 10.

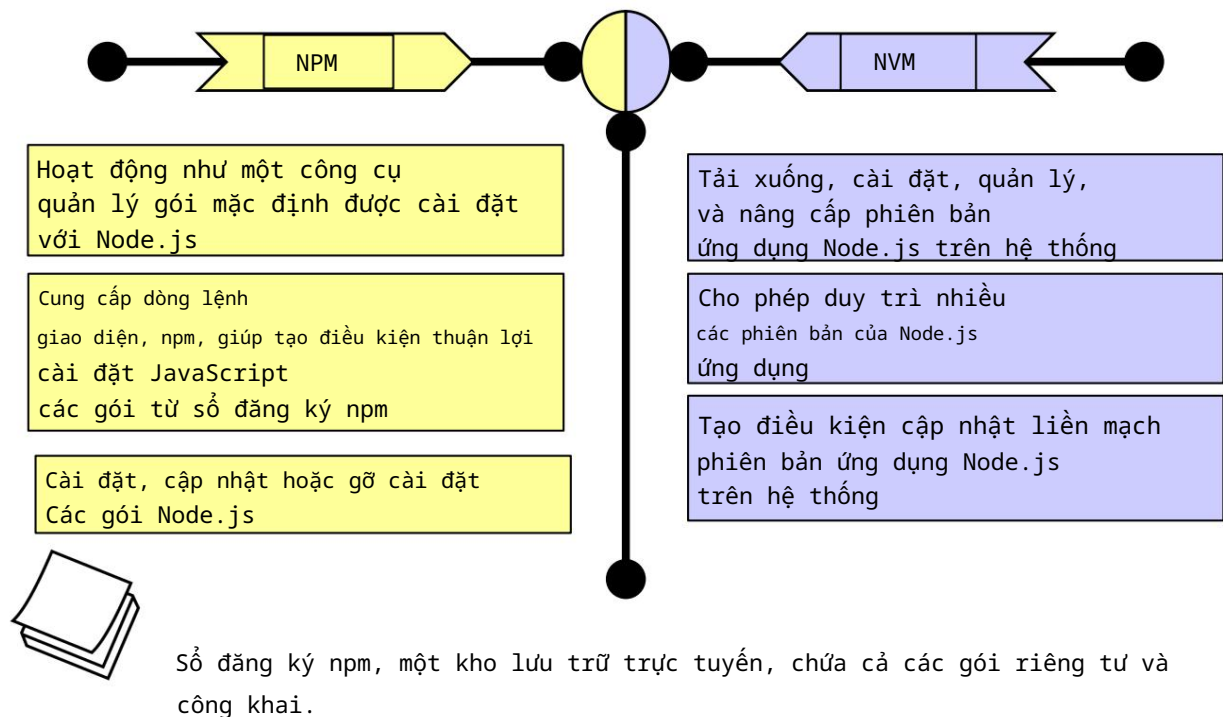
```
Average marks of five subjects: 84.6
```

Hình 2.15: Đầu ra của đoạn mã 10

Quan sát kết quả đầu ra của chương trình. Điểm trung bình được tính toán sẽ được hiển thị.

2,5 NPM và NVM

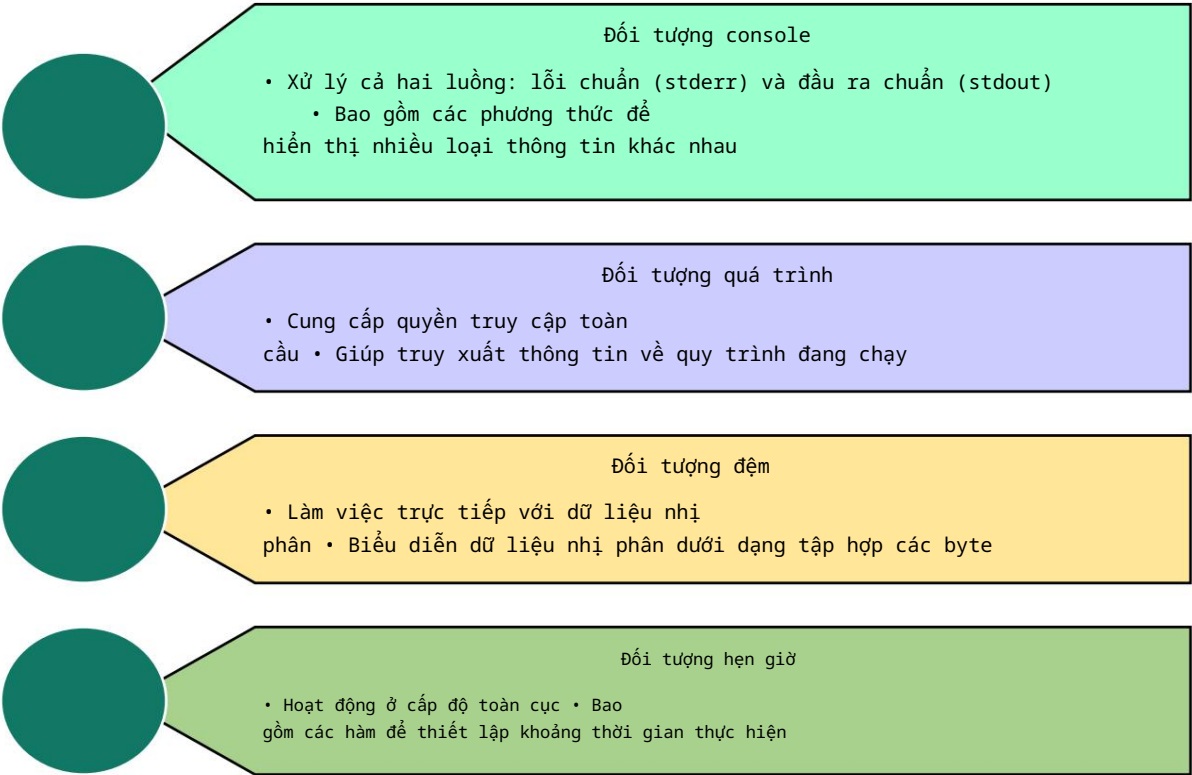
Node Package Manager (NPM) và Node Version Manager (NVM) là những công cụ quan trọng để quản lý phát triển Node.js và JavaScript.



2.6 Đối tượng toàn cục

Có một số đối tượng trong Node.js mang tính toàn cục. Những đối tượng này có thể được sử dụng trực tiếp trong các ứng dụng Node.js. Không bắt buộc phải import bất kỳ module nào trước khi sử dụng các đối tượng toàn cục.

Sau đây là một số đối tượng toàn cục thường được sử dụng trong Node.js:



Làm việc với Đối tượng Toàn cục

Sau đây là một số đoạn mã về các đối tượng toàn cục.

đối tượng điều khiển

Hãy xem xét một đoạn mã ví dụ trong Đoạn mã 11, trong đó đối tượng bảng điều khiển được minh họa.

```
Đoạn mã 11: const console
= require('console');

console.log('console.log được sử dụng để hiển thị một thông báo đơn giản');

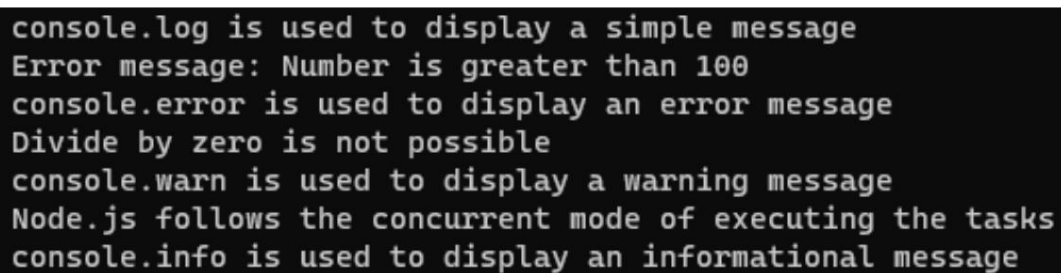
số = 100 nếu (số >
150) {
  console.log('Số nhỏ hơn 100'); } else { console.error('Thông báo lỗi:
Số lớn hơn
100'); }

console.error('console.error được sử dụng để hiển thị thông báo lỗi');
```

```
a = 110; b =  
0;  
a = a/b;  
  
hàm divbyzero(a, b) {  
  
  nếu (a == 0) {  
    console.warn('Kết quả là ${b}'); } else  
  
    { console.warn('Không thể chia cho số 0');  
  
  }  
} } divbyzero(a, b);  
  
console.warn('console.warn được sử dụng để hiển thị thông báo cảnh báo');  
  
console.info('Node.js tuân theo chế độ đồng thời để thực hiện các tác vụ');  
  
console.info('console.info được sử dụng để hiển thị thông báo thông tin');
```

Trong Code Snippet 11, nhiều phương thức khác nhau từ lớp console được đưa vào.
Mỗi phương pháp đều có chức năng riêng.

Hình 2.16 hiển thị kết quả của Đoạn mã 11.



```
console.log is used to display a simple message  
Error message: Number is greater than 100  
console.error is used to display an error message  
Divide by zero is not possible  
console.warn is used to display a warning message  
Node.js follows the concurrent mode of executing the tasks  
console.info is used to display an informational message
```

Hình 2.16: Đầu ra của đoạn mã 11

Quan sát đầu ra của chương trình. Các thông báo được hiển thị trên bảng điều khiển.
đối tượng quy trình

Hãy xem xét một đoạn mã ví dụ trong Đoạn mã 12, trong đó đối tượng quy trình được minh họa.

Đoạn mã 12:

```
console.log(`Thư mục hiện tại: ${process.cwd()}`);  
console.log(`Đối số dòng lệnh: ${process.argv}`);
```

Trong Đoạn mã 12, phương thức `process.cwd` được sử dụng để truy cập thư mục làm việc hiện tại của chương trình. Đối tượng `process argv` được sử dụng để in các đối số dòng lệnh được cung cấp cho chương trình Node.js bằng thuộc tính `process.argv`.

Hình 2.17 hiển thị kết quả của Đoạn mã 12.

```
C:\Node Js>node process_global.js  
Current directory: C:\Node Js  
Command-line arguments: C:\Program Files\nodejs\node.exe,C:\Node Js\process_global.js
```

Hình 2.17: Đầu ra của đoạn mã 12

Quan sát đầu ra của chương trình. Thư mục làm việc hiện tại và các đối số dòng lệnh được hiển thị.

Đối tượng đệm

Hãy xem xét một đoạn mã ví dụ trong Đoạn mã 13, trong đó đối tượng Buffer được minh họa.

Đoạn mã 13:

```
var buf_obj = Buffer.from('Học');  
console.log(buf_obj);
```

Trong Đoạn mã 13, chuỗi 'Học' được chuyển đổi thành dữ liệu nhị phân.

Hình 2.18 hiển thị kết quả của Đoạn mã 13.

```
<Buffer 4c 65 61 72 6e 69 6e 67>
```

Hình 2.18: Đầu ra của đoạn mã 13

Quan sát đầu ra của chương trình. Dạng nhị phân của dữ liệu chuỗi được hiển thị.

Đối tượng hẹn giờ

Hãy xem xét một đoạn mã ví dụ trong Đoạn mã 14, trong đó đối tượng Timer được minh họa.

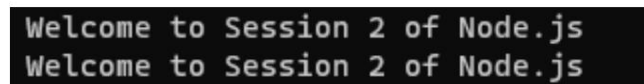
Đoạn mã 14:

```
hàm display() {  
  console.log('Chào mừng đến với Phiên 2 của Node.js');  
}  
const timeinterval = setInterval(display, 1000);  
setTimeout(() => {clearInterval(khoảng thời gian);}, 3000);
```

Trong Đoạn mã 14, phương thức `display` được thực thi mỗi giây. Phương thức `setInterval` thực thi phương thức `display` liên tục tính bằng mili giây, theo các khoảng thời gian được chỉ định.

Sau ba giây, phương thức `clearInterval` được gọi với ID khoảng thời gian được cung cấp bởi phương thức `setInterval`. Phương thức `clearInterval` chấm dứt việc thực thi liên tục của phương thức `display`.

Hình 2.19 hiển thị kết quả của Đoạn mã 14.



```
Welcome to Session 2 of Node.js  
Welcome to Session 2 of Node.js
```

Hình 2.19: Đầu ra của đoạn mã 14

Quan sát đầu ra của chương trình. Thông báo chỉ hiển thị hai lần khi chức năng hiển thị kết thúc.

2.7 Tóm tắt

Node.js, một môi trường chạy thời gian thực dựa trên JavaScript, có nhiều thành phần khác nhau để xử lý yêu cầu của máy khách, hoàn tất việc thực thi và cung cấp phản hồi.

Node.js quản lý nhiều yêu cầu của khách hàng cùng lúc.

Node.js là luồng đơn và tuân theo lập trình không đồng bộ người mẫu.

Môi trường REPL đọc dữ liệu đầu vào, đánh giá mã, in ra kết quả và lặp lại các tác vụ.

Các ứng dụng dựa trên bảng điều khiển thực hiện nhiệm vụ của chúng trong dòng lệnh môi trường.

NVM quản lý các phiên bản của ứng dụng Node.js.

NPM cài đặt, cập nhật và gỡ cài đặt các gói và mô-đun Node.js.

Các đối tượng toàn cục trong Node.js có sẵn trong suốt vòng đời của một ứng dụng.

Kiểm tra kiến thức của bạn

1. Chức năng chính của vòng lặp sự kiện trong Node.js là gì?

- a) Lưu trữ các yêu cầu của khách hàng
- b) Xử lý các yêu cầu của khách hàng bằng các hoạt động I/O chặn
- c) Xử lý các tài nguyên bên ngoài
- d) Gửi phản hồi cho khách hàng

2. Lợi ích nào sau đây là của việc xây dựng các ứng dụng phía máy chủ trong Node.js?

- a) Khả năng mở rộng dễ dàng
- b) Xử lý yêu cầu đồng thời
- c) Giới hạn luồng đơn
- d) Sử dụng bộ nhớ

3. Bạn sẽ sử dụng phương pháp nào để hiển thị văn bản đã cho trên bảng điều khiển?

Node.js là môi trường chạy hiệu quả dựa trên JavaScript trên bảng điều khiển.

- a) `console.display()`
- b) `console.write()`
- c) `console.log()`
- d) `console.message()`

4. Bạn sẽ sử dụng môi trường nào sau đây để kiểm tra nhanh một số Mã JavaScript?

- a) Node.js IDE
- b) Bảng điều khiển trình duyệt web
- c) REPL
- d) Dấu nhắc lệnh

5. Từ khóa nào được sử dụng để khai báo biến trong JavaScript?

- a) `let`
- b) `const`
- c) `def`
- d) `var`

Câu trả lời để kiểm tra kiến thức của bạn

1	
ngày 2	
a, b	
3 c	
4 c 5 d	

Hãy tự mình thử nhé

- Viết chương trình thực hiện các phép tính số học trên tập hợp số cho trước. Khởi tạo ba biến `x`, `y` và `z` với các giá trị 27, 7.2, và -3 tương ứng. Thực hiện các phép tính đã cho và hiển thị kết quả:
 - Hiển thị tổng của `x` và `z`
 - Hiển thị kết quả của phép trừ `y` khỏi `z`
 - Hiển thị tích của `z` và `y`
 - Hiển thị thương của `x` chia cho `y`
 - Hiển thị số dư khi chia `x` cho `y`
- Viết chương trình thực hiện các phép toán trên các giá trị boolean và các biến số. Khởi tạo một biến boolean `i` với giá trị `false` và hai biến số `x` và `y` với giá trị 10 và 1. Thực hiện các phép toán đã cho.
các hoạt động và hiển thị kết quả:
 - Hiển thị nếu giá trị của `x` giống với giá trị của `y`
 - Kiểm tra và hiển thị nếu giá trị của `x` không giống với giá trị của `y`
 - Hiển thị giá trị boolean của phủ định của biến `i`
- Viết chương trình thực hiện nối chuỗi. Khởi tạo hai biến chuỗi, `text1` với giá trị `Welcome to` và `text2` với giá trị `JavaScript Programming`. Thực hiện các phép toán cho sẵn và hiển thị kết quả:
 - Hiển thị giá trị của biến `text1`
 - Hiển thị giá trị của biến `text2`
 - Nối hai chuỗi `text1` và `text2` và hiển thị kết quả
- Viết chương trình hiển thị thông tin về các cầu thủ cricket. Định nghĩa một đối tượng JavaScript tên là `cricket` với các thuộc tính như `Name` (Tên), `Run` (Chạy) và `Wickets` (Cầu thủ). Thực hiện các thao tác cho sẵn và hiển thị kết quả:
 - Hiển thị tên của cầu thủ `cricket`
 - Hiển thị đường chạy của cầu thủ `cricket`
 - Hiển thị số lượng `Wickets` được đội thực hiện

5. Tạo một hàm có tên là `salesPersonInMarketing` để lấy

các tham số đã cho:

`tên`: Tên của nhân viên bán hàng.

`tuổi`: Độ tuổi của nhân viên bán hàng.

`kinh nghiệm`: Số năm kinh nghiệm trong lĩnh vực bán hàng.

`salesInQuarter1`: Doanh số đạt được trong quý đầu tiên.

`salesInQuarter2`: Doanh số đạt được trong quý 2.

Hàm này sẽ thực hiện các phép toán đã cho và hiển thị kết quả:

a. Hiển thị tên của nhân viên bán hàng

b. Hiển thị tuổi của nhân viên bán hàng

c. Hiển thị số năm kinh nghiệm trong lĩnh vực bán hàng

d. Tính toán và hiển thị tổng doanh số đạt được trong cả hai quý

6. Viết một chương trình để tính điểm kiểm tra trung bình của một cầu thủ cricket qua năm trận đấu thử nghiệm tại World Cup. Bạn có một mảng tên là `testScoresMatch` chứa điểm kiểm tra của cầu thủ này.

Tính điểm kiểm tra trung bình và hiển thị kết quả.

a. Tạo một mảng `testScoresMatch` với điểm kiểm tra của một cầu thủ cricket bằng cách sử dụng dữ liệu này `[111, 55, 303, 324, 400]`

b. Tính tổng điểm các bài kiểm tra

c. Tính toán và hiển thị điểm kiểm tra trung bình